

Guia de Padronização Git e GitHub

Criado por: Heitor Santos Nobre

1. Hierarquia de Branches

Para garantir a estabilidade do projeto, adotamos a seguinte estrutura de ramificações:

- **main:** É a branch principal. Contém o código estável, testado e pronto para produção. Nada deve ser desenvolvido diretamente aqui.
- **develop:** É a branch de integração. Recebe todas as novas funcionalidades e correções. É nela que validamos se o sistema está funcionando como um todo antes de subir para a `main`.
- **Branches de Tarefa:** Criadas por cada desenvolvedor para trabalhar em demandas específicas. Elas nascem a partir da `develop` e retornam para ela via Pull Request.

2. Nomenclatura de Ramificações

- `feature/nome-recurso` : Implementação de novas funcionalidades.
- `fix/nome-correcao` : Resolução de bugs.
- `docs/ajuste-texto` : Documentação.

3. Padrão de Commits (Conventional Commits)

Utilize prefixos para descrever o tipo de alteração:

- `feat:` novas funcionalidades. | `fix:` correção de erros.
- `chore:` infraestrutura/dependências. | `refactor:` melhoria de código.

```
git commit -m "feat: implementar lógica de cálculo de impostos"
```

4. Comandos Essenciais

Comando	Utilidade
<code>git status</code>	Verifica arquivos modificados.
<code>git add .</code>	Prepara arquivos para o commit.
<code>git pull origin develop</code>	Atualiza seu código com a branch de integração.
<code>git push origin [branch]</code>	Envia seu trabalho para o servidor.
<code>git stash / pop</code>	Salva/Recupera alterações temporárias.
<code>git checkout [branch]</code>	Alterna entre ramificações.

5. Fluxo de Trabalho Passo a Passo

1. Garanta que sua `develop` está atualizada: `git checkout develop` e `git pull`.
2. Crie sua branch de tarefa: `git checkout -b feature/minha-tarefa`.
3. Desenvolva, adicione os arquivos e faça commits seguindo o padrão.
4. Envie sua branch: `git push origin feature/minha-tarefa`.
5. Abra um **Pull Request (PR)** para a branch `develop`.
6. Após aprovado e testado na `develop`, o responsável fará o merge para a `main`.

Dica: Nunca faça merge de uma branch de tarefa diretamente na `main`. O fluxo correto é sempre passar pela `develop` primeiro para garantir que sua alteração não quebra o trabalho dos colegas.

6. Configuração e Início

```
# Identificação do autor
git config --global user.name "Seu Nome"
git config --global user.email "seu.email@exemplo.com"

# Iniciar projeto
git init
```

7. Considerações Finais

Este fluxo garante que a `main` permaneça sempre funcional e que a `develop` sirva como um ambiente seguro de testes coletivos. Mantenha o repositório organizado.